

Lecture 9: HTTP Servers and APIs

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- Dynamic web pages
- Browsers, forms and server-side scripting
- HTTP APIs
 - Form/query, RPC and REST styles
- Scaling up HTTP services

Dynamic web pages

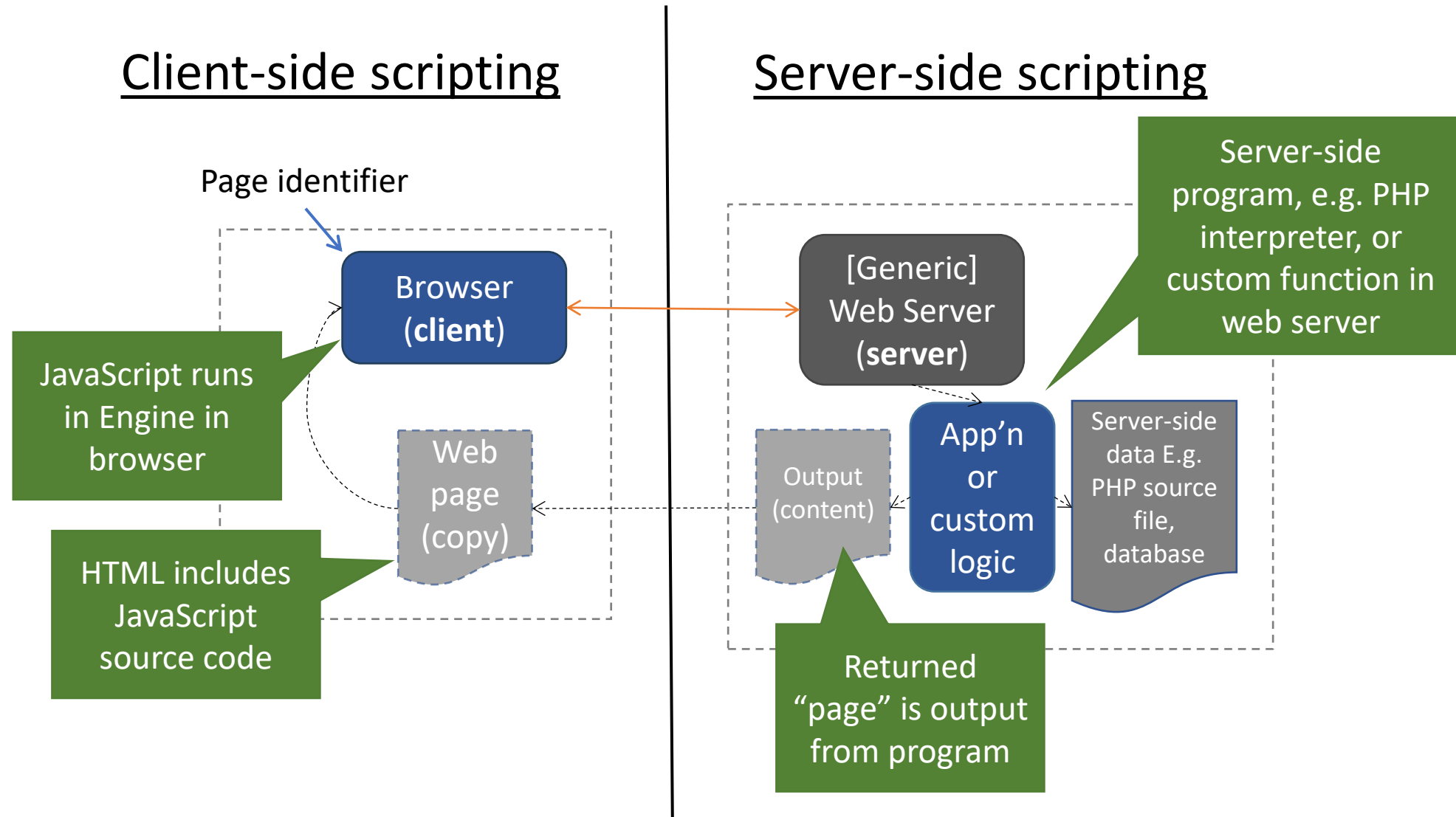
Server-side scripting

- The HTML that a browser receives may be
 - copied verbatim from a file on the server, a.k.a. “**static**”, or
 - **Dynamically generated**, i.e. generated on-the-fly by running code on the server to handle the request
- The latter is called **Server-Side Scripting**
 - Can be done in many different languages, e.g. PHP, Javascript (in Node), C#/.NET, go, ...
 - The code executes on the **server**
 - The HTML is the output of the code
- Some languages (e.g. PHP) can mix HTML text and server-side code in the same file
 - Often called templating

Client-side Scripting

- An HTML page can include embedded **Javascript**
 - In **<script>** elements
 - So the code text is transferred to the browser
- This code is run in the **browser** (i.e. client)
 - Not on the server!
- It can (e.g.)
 - Handle user input events
 - Make HTTP requests
 - Dynamically update the HTML
 - Be used on its own, or combined with server-side scripting...

Client-side vs Server-side Scripting



Browsers, forms and **server-side** scripting

(old dynamic web 😊)

HTTP from a browser with traditional server-side scripting (GET URL)

- **User**: Enter a **URL** or click a **link** => **Browser** HTTP **GET** on that URL
 - **Server**: URL typically identifies a server-side file, which is either
 - (a) a simple file, to be copied straight back to client, OR
 - (b) a **script/program** which corresponds to a specific **operation**:
 - Server runs the script
 - Query parameters in URL & request body -> arguments/inputs for the server-side script
 - The script output -> the response body
 - **Browser**: displays (or downloads) the returned body, or is redirected to a new URL (by a redirect in the response header)
 - If displayable (e.g. HTML) it replaces the current document/page

HTTP from a browser with traditional server-side scripting (Form)

Q: Examples?
At UoN?

- **Server** sends HTML **form** to **Browser** in current HTML page:
 - **Browser**: renders <form> element
 - Specifies submit method (GET or POST) and URL
 - with various <input> elements where user can enter information
 - E.g. text fields, checkboxes, file selectors, drop-downs, and Submit button
- **User**: Press form **submit** => **Browser** HTTP POST (or GET) on URL
 - Form input values encoded -> request body
 - Either application/x-www-form-urlencoded or multipart/form-data
 - **Server**: HTTP request handled as previous slide
 - i.e. URL path -> script/program to run, with form values as input; output -> response body sent to client
 - **Browser** displays (or downloads) the returned body, or is redirected to a new URL, replacing the form document/page

E.g. HTML form

- Lab 5 <https://people.cs.nott.ac.uk/pszcmg/G52DSY/form.html>

```
<form action="test2.php"  
  enctype="application/x-www-form-urlencoded"  
  method="post">
```

Name:

```
<input type="text" name="name">
```

```
<input type="submit" value="submit">
```

```
</form>
```

Name:

Form operation

- Press submit => HTTP POST:

URL:

`https://people.cs.nott.ac.uk/pszcmg/G52DSY/test2.php`

server

operation -> file (script)

Request body:

`name=123`

Arguments in request body

Response body

= result (plain text): `hello 123, your name is 3 character(s) long`

HTTP APIs

HTTP Application Programming Interfaces (APIs)

- HTTP is (now) often used as a standard transport mechanism for program-to-program interaction,
- Designed as an interface to a service, not as a website (HTML/etc.)
 - Still HTTP, client/server, request/response, ...
 - But not (necessarily) requests that a (simple) browser could ever produce
 - And not usually HTML responses, e.g. JSON, XML
- Used by:
 - Custom clients (in any programming language),
 - including other servers, and
 - JavaScript in browsers (i.e. AJAX [although the X is not usually XML any more])

HTTP APIs Specifications

- Usually language-independent
 - i.e. defined in terms of HTTP and common data encodings, not specific programming language types or interfaces
- Normally defined in terms of permitted combinations of
 - URL, esp. path
 - And any query parameters
 - HTTP method
 - Authentication requirements
 - Any other header requirements
 - Request body encoding & content
 - Response body encoding & content
 - Error responses, i.e. HTTP status codes and/or response body values

Defining HTTP APIs

- Can be defined/specified
 - By manual documentation
 - Created before or after the code
 - Automatically from the server implementation
 - E.g. by example, reflection and/or programming language annotations
 - In a machine-readable interface definition / specification
 - E.g. SOAP (XML Web Services)
 - Swagger / OpenAPI specification (more recent/contemporary) – see next lecture

Styles of HTTP API

- HTTP can be used in several different ways or styles (see following slides...):
 - Form- & query-style
 - (might be used from regular web pages without JavaScript)
 - RPC (Remote Procedure Call)-style
 - REST (REpresentational State Transfer)
 - Or a mixture of the above
 - (Or other including arbitrary non-standard/proprietary approaches)
 - E.g. if HTTP is just being used to get past firewalls rather than for interoperability

HTML Form- or Query-style operations

Designed to be easily supported by server-side scripting frameworks developed for regular browser client (e.g. PHP):

- **URL** defines the server and **operation** (path)
- HTTP method is **GET** (for normal navigation or data requests without side-effects) or **POST** (esp. for operations with side-effects)
- Arguments are a list of **names and text values** (or, if multipart, files)
 - In the URL **query parameters** (GET) or
 - In the request body (Form POST), encoded either
 - Like a query string (MIME type `application/x-www-form-urlencoded`)
 - Like a MIME multipart message (`multipart/form-data`)
- Response body is **machine readable** result (i.e. not usually HTML)
- Errors may be signalled by HTTP status and/or response body

E.g. query-style

See
<https://developers.google.com/maps/documentation/geolocation/get-api-key>

- Google maps API

- <https://developers.google.com/maps/documentation/geocoding/start>
- E.g. look up an address, GET:

```
https://maps.googleapis.com/maps/api/geocode/json?address=1600+Amphitheatre+Parkway,+Mountain+View,+CA&key=YOUR_API_KEY
```

server

operation

Arguments in query parameters

Response body
= result
(JSON-encoded):

```
{ "results" : [{ ...  
    "geometry" :  
    { "location" :  
        { "lat" : 37.4224764,  
          "lng" : -122.0842499 } ... } ... }],  
  "status" : "OK"  
}
```

Remote Procedure Call (RPC)

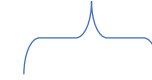
Starts with RPC concepts (e.g. interfaces, methods), NOT HTTP:

- URL typically identifies a specific **service**
 - May also identify the operation
- HTTP method is normally **POST**
- **Body of request** encodes the specific **operation** name if not specified by URL and a list of **arguments**
 - cf. request/reply protocol message
 - The body encoding is specified by choice of RPC technology
- **Response body** encodes **status** and **results** of operation
- Errors normally signalled within the response body

E.g. RPC-style

- E.g. Simple Object Access Protocol (SOAP)
 - <https://www.w3.org/TR/soap/>
 - E.g. GetStockPrice(IBM) request:

Service



```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml;
charset=utf-8
Content-Length: 299
SOAPAction:
"http://www.w3.org/2003/05/soap-envelope"
```

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
xmlns:m="http://www.example.org/stock/Surya">
  <soap:Header>
  </soap:Header>
  <soap:Body>
    <m:GetStockPrice>
      <m:StockName>IBM</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

Operation
Arguments



```
<m:GetStockPrice>
  <m:StockName>IBM</m:StockName>
</m:GetStockPrice>
```

REST – Representational State Transfer

Starts with HTTP concepts, i.e. resources (URLs) & HTTP methods:

- URL identifies a specific **resource**, e.g. a database record
 - In practice URL path often include path elements representing:
Resource type; resource identifier; view name, facet or relationship
- **HTTP method** identifies operation to perform
 - GET = read, PUT = write, POST = create/append, DELETE = delete, PATCH = edit
- For GET the **response body** (result) is an external data representation of the identified resource
 - E.g. in **JSON or XML encoding** as specified by content-type header
- Query parameters may be used IF the resource URL is not specific enough (e.g. search rather than simple get)
 - For **PUT/POST** the only argument – the **request body** – is a (new) representation of the identified/new resource
- Errors are signalled by **HTTP status** whenever possible

E.g. REST-style

See
<https://developer.spotify.com/documentation/web-api/reference/get-an-artist?id=> (requires OAuth bearer token in Authorization header)

- Spotify API
 - <https://developer.spotify.com/documentation/web-api>
 - E.g. get information about an artist, GET:

`https://api.spotify.com/v1/artists/0OdUWJ0sBjDrqHygGUXeCF`

server

API

Entity
type

Artist ID (resource)

Response body
= representation of resource
(JSON-encoded):

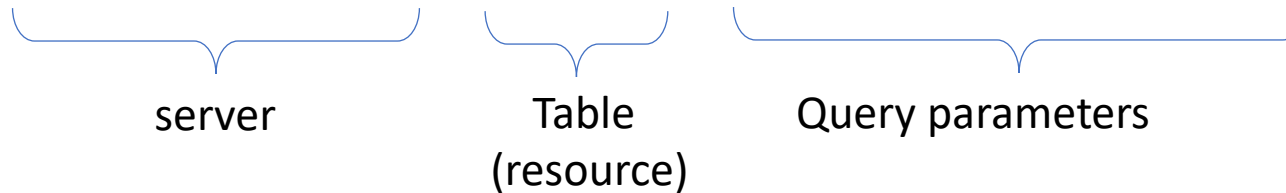
```
{ ...  
  "genres" : [ "chamber pop", "folk-pop", ... ],  
  "id" : "0OdUWJ0sBjDrqHygGUXeCF",  
  "name" : "Band of Horses", ...  
}
```

E.g. REST-style (course-grained)

See https://docs.postgrest.org/en/v14/references/api/tables_views.html
(requires signed client-identifying bearer token [JWT] in Authorization header if not local)

- Supabase REST API
 - <https://docs.postgrest.org/en/v14/index.html>
 - E.g. rows from “people” table with “age” less than 13, GET:

`http://HOST:PORT/people?age=lt.13`



Response body
= representation of resource
(JSON-encoded):

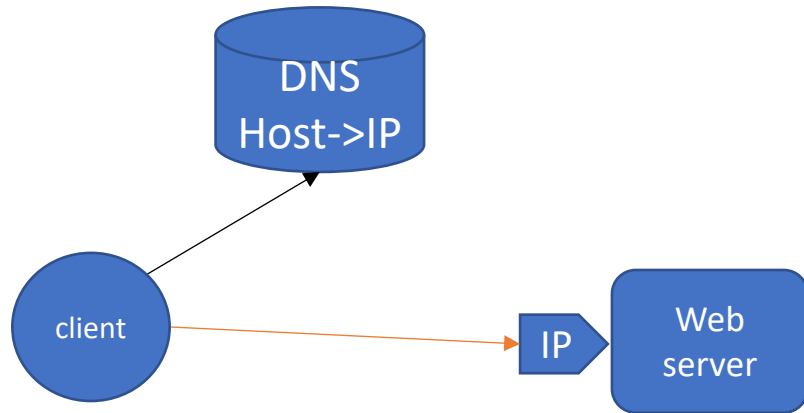
```
[  
  {"first_name": "John", "age": 10},  
  {"first_name": "Jane", "age": 12}  
]
```

Q: have you used any other web APIs?

Scaling up HTTP services

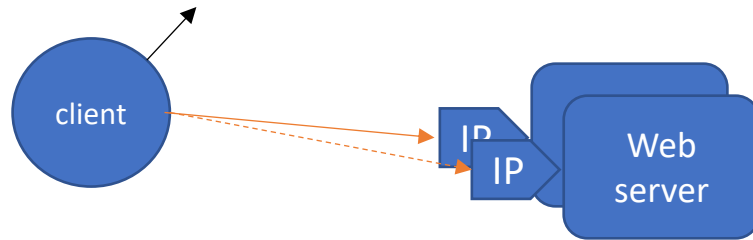
A few notes on scaling up (1)

1.



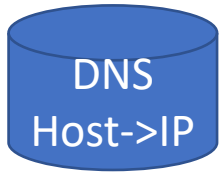
1. Purpose => domain name => Application-specific IP

2.

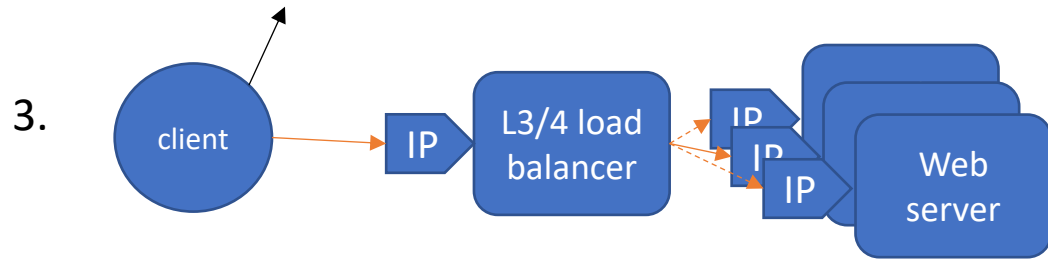


2. Single domain name => multiple IPs (DNS)

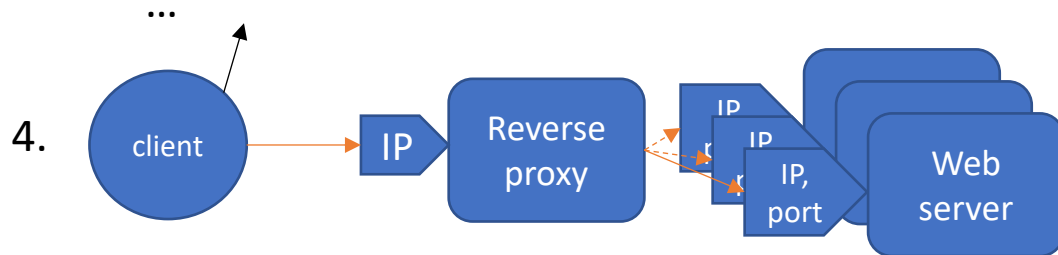
- E.g. replicated servers (e.g. BBC website in DNS lab)



A few notes on scaling up (2)



- 3. Single IP => layer 3/4 load balancer => multiple hosts
 - i.e. uses only IP/ports to balance
 - Can combine with the above
 - Can dynamically scale or replace servers (VMs or containers)



- 4. Single host => reverse proxy (as layer 5 load balancer) => multiple hosts
 - i.e. can also use URLs etc to balance or (more often) assign requests to specific back-end services
 - Can combine with the above

Content Distribution Networks (CDNs) and anycast

- Content Distribution Networks use **anycast**...
 - They have servers at several data centres around the world
 - Sometimes called “cloud edge”
 - Servers in different data centres have the SAME public IP addresses
 - The global routing protocol (BGP) lets routers direct traffic to the CLOSEST server with that IP address
 - (They also have own unique IP addresses for internal use)
- Content can be pushed out to all CDN servers and served from a local cache
- And/or CDN servers can **proxy** HTTP requests,
 - To a specific IP address, i.e. your server (and cache responses), or
 - To a different IP address depending where in the world they are
 - i.e. to one of several servers around the world running your code

E.g. see <https://developers.cloudflare.com/learning-paths/get-started/concepts/how-cloudflare-works/>

Dynamic Web Pages and Scripting

Summary

- Most web pages use **HyperText Markup Language** (HTML) which specifies the representation of a document for display in a browser, but...
- Responses can be generated in any way, e.g. **dynamic** pages
 - Generated by **server-side scripts** or custom server
- HTML page (sent by the server) can include **JavaScript** to be run in the browser
 - i.e. **client-side scripts**, which directly change the browser view & can directly make requests to (same or other) server(s)
 - (And note HTTP can transport any type of content, not just HTML)

HTTP APIs Summary

- HTTP Application Programming Interfaces (**APIs**) can be defined in various ways
 - Generally specifies combinations of URL, HTTP method, authentication, body, response and error signalling
- HTTP can be used in different ways (idioms) in a distributed system, e.g.
 - **Form- & query**-style - Designed to be easily supported by server-side scripting frameworks developed for regular browser client (e.g. CGI, PHP)
 - **RPC** (Remote Procedure Call)-style - Starts with RPC concepts (e.g. interfaces, methods), NOT HTTP
 - **REST** (REpresentational State Transfer) - Starts with HTTP concepts, i.e. resources (URLs) & HTTP methods

Scaling up HTTP services summary

- Various strategies can be used (and combined) to distribute the load of a single logical HTTP server over multiple machines
 - E.g. multiple DNS A records, L3/4 load balancers, L5 load balancers, anycast and CDNs

Next steps

- Lab 5: command-line and browser HTTP
- Lecture 10: Example HTTP APIs, Clients and Servers
- Lab 6 (next week): Java HTTP